

Học Sâu (Deep Learning)

Chương 2: Nền tảng Toán học của Mạng Nơ-ron

Đại học Đông Á

July 23, 2026

Nội dung chương

1. Ví dụ đầu tiên về Mạng Nơ-ron (MNIST)
2. Biểu diễn dữ liệu bằng Tensor
3. Phép toán Tensor (Tensor Operations)
4. Tối ưu hóa Dựa trên Gradient
5. Đồ thị Tính toán & Lan truyền ngược
6. Tổng kết

Ví dụ kinh điển: Tập dữ liệu MNIST

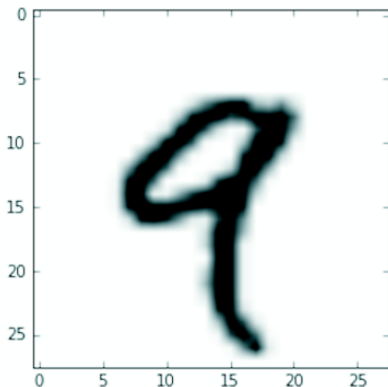
- **Bài toán:** Phân loại ảnh các chữ số viết tay (từ 0 đến 9).
- **Dữ liệu:** 60.000 ảnh huấn luyện, 10.000 ảnh kiểm tra.
- Kích thước mỗi ảnh: 28×28 pixels (ảnh xám đen trắng).
- Trong học máy:
 - Phân loại (Category) gọi là **Lớp (Class)**.
 - Dữ liệu đầu vào gọi là **Mẫu (Sample)**.
 - Kết quả của mẫu gọi là **Nhãn (Label)**.



Hình 2.1: Một số mẫu ảnh từ MNIST

Xem xét một mẫu dữ liệu (Sample)

- Mỗi bức ảnh là một ma trận số nguyên.
- Giá trị từ 0 (trắng) đến 255 (đen).
- Ở đây chúng ta hiển thị mẫu thứ 4 trong tập huấn luyện (chữ số '4').



Hình 2.2: Biểu diễn ma trận của một ảnh chữ số '4'

Định nghĩa Tensor (Bộ căng)

Trong học sâu, mọi dữ liệu đều được biểu diễn thông qua các mảng đa chiều gọi là **Tensor**. Số lượng trục (axes) của tensor được gọi là Hạng (Rank).

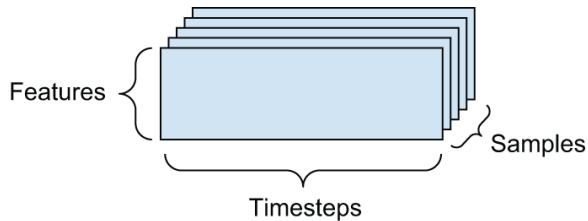
- **Vô hướng (Scalar) - Tensor 0D:** Chỉ chứa một con số duy nhất. Ví dụ: $x = 12$.
- **Vector - Tensor 1D:** Một mảng các con số. Trục duy nhất biểu diễn số chiều của vector.
- **Ma trận (Matrix) - Tensor 2D:** Một mảng của các vector (có Hàng và Cột).
- **Tensor 3D:** Xếp các ma trận thành hình hộp.

Tensors trong thực tế: Dữ liệu Chuỗi thời gian

Dữ liệu chuỗi thời gian (Timeseries) hoặc chuỗi trình tự thường được biểu diễn bằng **Tensor 3D**.

- Chiều 1: Mẫu (Samples)
- Chiều 2: Thời gian (Timesteps)
- Chiều 3: Đặc trưng (Features)

Ví dụ: Giá cổ phiếu mỗi phút (giá hiện tại, giá cao nhất, giá thấp nhất) thu thập trong 390 phút mỗi ngày.

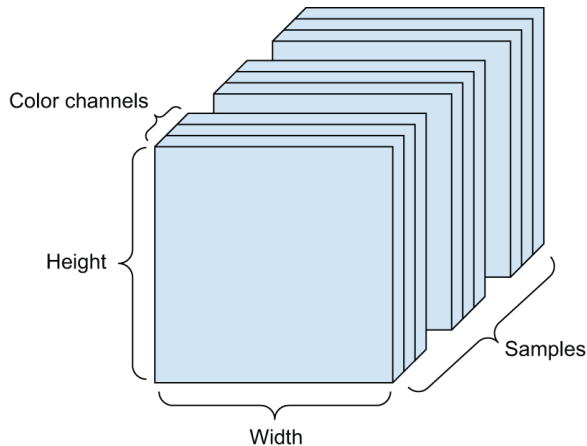


Hình 2.3: Tensor 3D cho dữ liệu chuỗi thời gian

Tensors trong thực tế: Dữ liệu Ảnh

Ảnh kỹ thuật số thường có 3 chiều: Chiều cao, Chiều rộng và Kênh màu (Color channels).

- **Lô ảnh (Batch of images):** Trở thành một **Tensor 4D**.
- *Ví dụ:* Một lô 128 bức ảnh màu cỡ 256×256 có hình dạng (Shape) là $(128, 256, 256, 3)$.



Hình 2.4: Tensor 4D biểu diễn dữ liệu ảnh màu

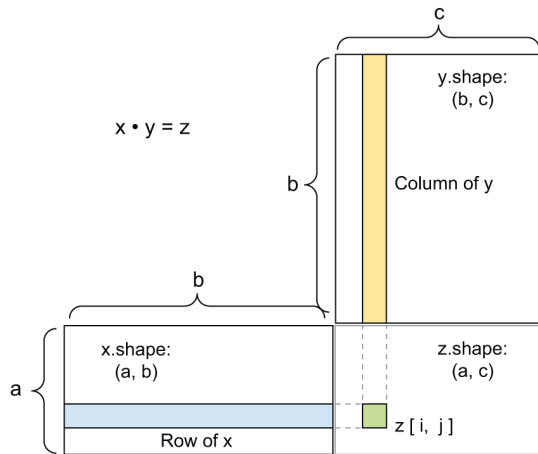
Tính chất Toán học

Trong học sâu, mọi phép biến đổi tại layer đều có thể quy về:

$$\text{output} = \text{relu}(W \cdot \text{input} + b) \quad (1)$$

Bao gồm 3 phép toán tensor cơ bản:

- 1 Phép nhân ma trận (*dot product*): $W \cdot \text{input}$
- 2 Phép cộng (*addition*): $+b$
- 3 Phép toán trên từng phần tử (*relu*): hàm $f(x) = \max(x, 0)$

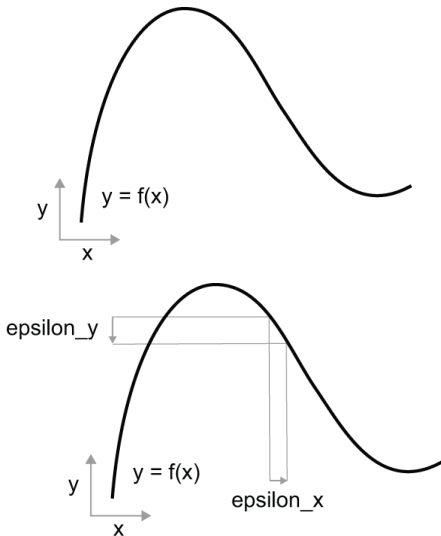


Hình 2.5: Sơ đồ minh họa phép nhân ma trận (Dot product)

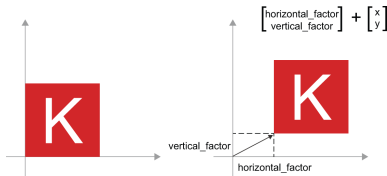
Ý nghĩa Hình học (1): Hàm số và Tính liên tục

Mỗi phép toán tensor đều biến đổi không gian hình học.

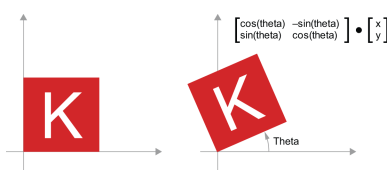
- Một mạng nơ-ron thực chất là một **hàm toán học (function)**.
- Hàm số này phải **liên tục (continuous)** và **trơn (smooth)**.



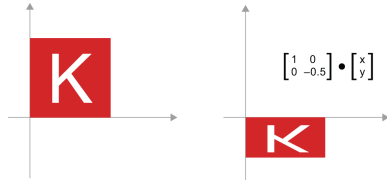
Ý nghĩa Hình học (2): Tịnh tiến, Phóng to, Quay



Hình 2.7: Tịnh tiến (Translation): Cộng vector



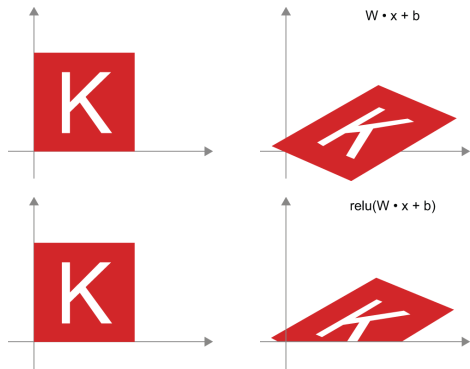
Hình 2.8: Quay (Rotation): Phép nhân ma trận



Hình 2.9: Thu phóng (Scaling): Nhân vô hướng

Ý nghĩa Hình học (3): Phép biến đổi Affine & Dense layer

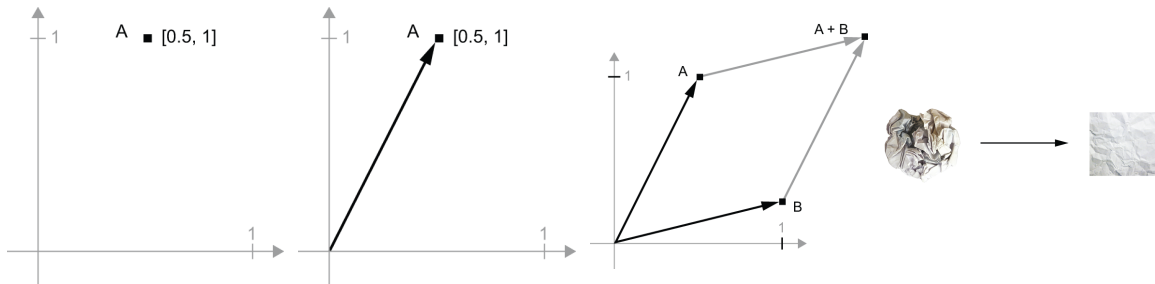
- **Phép biến đổi Affine** là sự kết hợp của một phép biến đổi tuyến tính (nhân ma trận) và một phép tịnh tiến (cộng vector).
- Đó chính xác là những gì diễn ra bên trong một lớp 'Dense': $y = W \cdot x + b$.



Hình 2.10: Biến đổi không gian qua lớp Dense

Ý nghĩa Hình học (4): Gỡ rối dữ liệu

Học sâu bản chất là chuỗi biến đổi hình học tinh vi để "gỡ rối" (untangle) các lớp dữ liệu đan xen nhau thành dạng có thể phân tách tuyến tính.

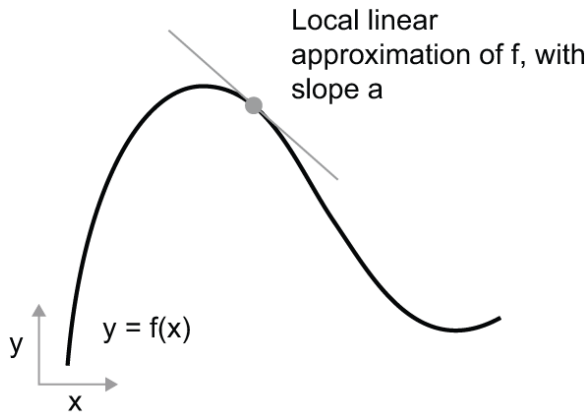


Hình 2.11: Quá trình gỡ rối không gian dữ liệu bằng học sâu giống như việc vuốt phẳng một quả bóng giấy vò nát.

Khái niệm Đạo hàm (Derivation)

Để điều chỉnh trọng số (Weights) một cách có định hướng, ta sử dụng **Đạo hàm**.

- Đạo hàm $f'(x)$ thể hiện **độ dốc (slope)** của tiếp tuyến với hàm số $f(x)$ tại điểm x .
- Nếu $f'(x) > 0$, ta giảm x thì $f(x)$ sẽ giảm.
- Nếu $f'(x) < 0$, ta tăng x thì $f(x)$ sẽ giảm.



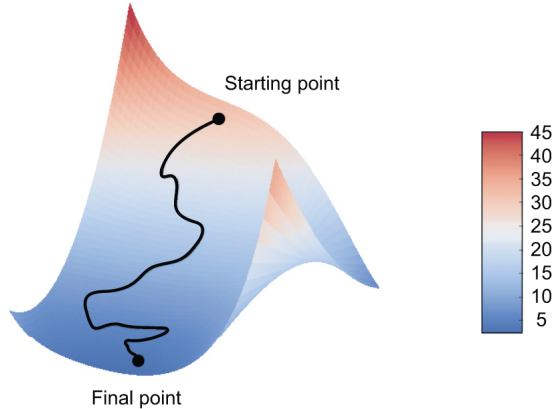
Hình 2.12: Đạo hàm là hệ số góc của tiếp tuyến cục bộ

Gradient và Mặt cắt Không gian Đa chiều

Trong mạng nơ-ron, hàm mất mát (Loss) phụ thuộc vào hàng triệu trọng số. Đạo hàm trong không gian đa chiều gọi là **Gradient**.

$$\nabla L(W) = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots \right) \quad (2)$$

Vectơ Gradient luôn chỉ về hướng làm cho hàm L **tăng nhanh nhất**.

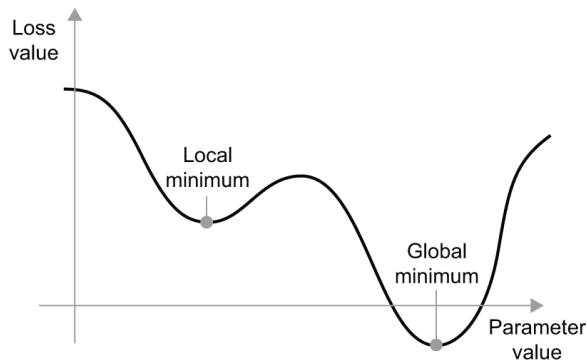


Hình 2.13: Mặt cong 3D của Loss function và Vector Gradient

Cực tiểu Toàn cục (Global Minimum)

Mục tiêu của tối ưu hóa là tìm **Cực tiểu toàn cục (Global Minimum)**, nơi mà Loss đạt giá trị nhỏ nhất tuyệt đối.

- Tại cực tiểu, Gradient bằng 0 ($\nabla L = 0$).
- Tuy nhiên, với hàm phức tạp, rất dễ bị kẹt tại cực tiểu cục bộ (Local Minimum).



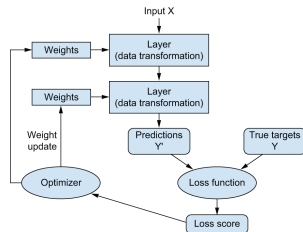
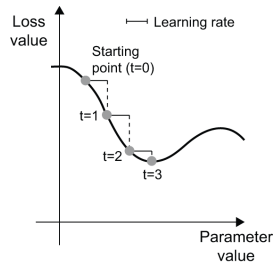
Hình 2.14: Local Minimum vs Global Minimum

Thuật toán Stochastic Gradient Descent (SGD)

SGD (Hạ Gradient Ngẫu nhiên): Thay vì tính chính xác đạo hàm trên toàn bộ dữ liệu, ta dùng 1 lô nhỏ (mini-batch) để ước tính.

$$W \leftarrow W - \text{step} \times \nabla L_{\text{batch}}(W) \quad (3)$$

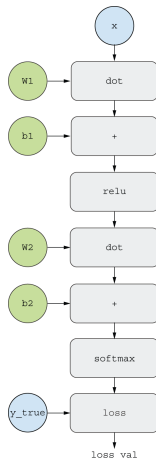
- Bước đi ngược hướng Gradient (âm) giúp trượt dần xuống thung lũng của hàm Loss.



Đồ thị Tính toán (Computation Graph)

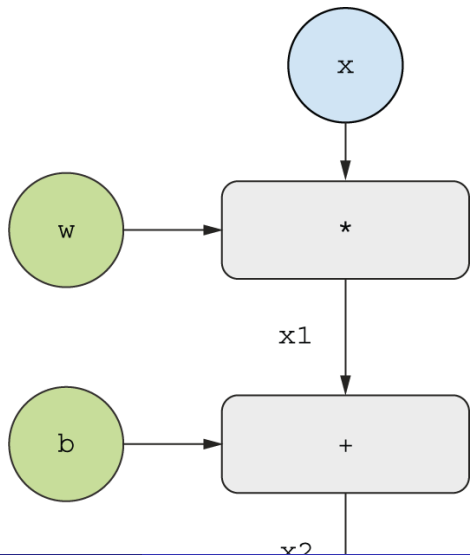
Mạng nơ-ron là một hàm ghép phức tạp. Để máy tính tính đạo hàm dễ dàng, nó được biểu diễn thành **Đồ thị tính toán**.

- Mỗi **nút (node)** là một phép toán tensor cơ bản (nhân, cộng, relu).
- Các **cạnh (edges)** dẫn giá trị truyền qua lại.



Hình 2.16: Đồ thị tính toán biểu diễn 1 Layer

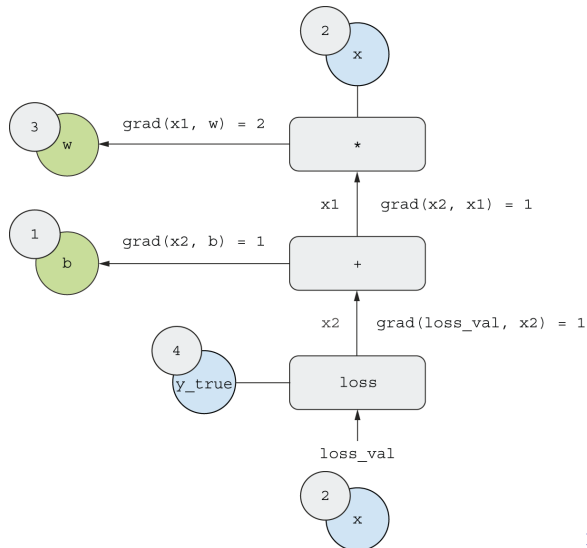
Ví dụ lan truyền thuận (Forward Pass)



Lan truyền ngược (Backpropagation)

Làm sao để tính đạo hàm của y theo x ? Dựa vào **Quy tắc chuỗi (Chain Rule)**:

$$\frac{df(g(x))}{dx} = f'(g(x)) \cdot g'(x) \quad (4)$$



Tổng kết Chương 2

- **Tensors:** Nền tảng cấu trúc dữ liệu của học sâu. Bao gồm các bậc 0D, 1D, 2D, 3D, 4D.
- **Tensor Operations:** Các phép nhân ma trận, phép cộng, phép biến đổi phi tuyến (ReLU)... là những biến đổi hình học giúp gỡ rối đa diện dữ liệu.
- **Gradient & SGD:** Đạo hàm giúp ta định hướng đi xuống dốc của Loss. SGD liên tục trượt theo Gradient để giảm Loss.
- **Backpropagation:** Sử dụng Đồ thị tính toán và Quy tắc chuỗi (Chain Rule) để phân phối ngược hàm lượng lỗi về từng trọng số, giúp mô hình tự tối ưu.